

```
In [34]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

In [142...]

df

Out[142...]

	Jersey No	Player	Matches	Inns	Not Out	Runs	Highest Score	Avg	Balls faced	Strike rate	50s
0	1	KL Rahul	14	14	2	670	132*	55.83	518	129.34	1
1	2	Shikhar Dhawan	17	17	3	618	106*	44.14	427	144.73	2
2	3	David Warner	16	16	2	548	85*	39.14	407	134.64	0
3	4	Shreyas Iyer	17	17	2	519	88*	34.60	421	123.27	0
4	5	Ishan Kishan	14	13	4	516	99	57.33	354	145.76	0
...	...	...	...	...	...	...	...	...	...	...	...
128	129	Khaleel Ahmed	7	1	0	0	0*	0.00	2	0.00	0
129	130	Arshdeep Singh	8	1	0	0	0*	0.00	3	0.00	0
130	131	Daniel Sams	3	1	0	0	0*	0.00	2	0.00	0
131	132	Shreevats Goswami	2	2	0	0	0*	0.00	4	0.00	0
132	133	Trent Boult	15	1	0	0	0*	0.00	1	0.00	0

133 rows × 18 columns



Load the dataset

```
In [3]: file_path = "C:/Users/praga/Downloads/IPL Data Set.xlsm"
xls = pd.ExcelFile(file_path)
df = pd.read_excel(xls, sheet_name="IPL_Dataset")
```

Clean column names

```
In [144...]: # Data cleaning with the columns names and mapping values
df.columns = df.columns.str.strip()
df.rename(columns={df.columns[10]: "100s", df.columns[11]: "50s"}, inplace=True)
print(df.columns)
```

```
Index(['Jersey No', 'Player', 'Matches', 'Inns', 'Not Out', 'Runs',
      'Highest Score', 'Avg', 'Balls faced', 'Strike rate', '100s', '50s',
      '6s', 'First_Name', 'Last_Name', 'Cleaned_Highest_score',
      'Cleaned_Highest_Score', 'Boundary_Percentage'],
      dtype='object')
```

Q1. What is the maximum number of matches played by an individual player in a season?
Print the player name along with the number of matches played.

```
In [8]: max_matches = df["Matches"].max()
player_max_matches = df[df["Matches"] == max_matches][["Player", "Matches"]]
print("Q1 Answer:")
print(player_max_matches)
```

Q1 Answer:

	Player	Matches
1	Shikhar Dhawan	17
3	Shreyas Iyer	17
19	Marcus Stoinis	17
67	Kagiso Rabada	17

```
In [125... print(df.columns)
```

```
Index(['Jersey No', 'Player', 'Matches', 'Inns', 'Not Out', 'Runs',
      'Highest Score', 'Avg', 'Balls faced', 'Strike rate', '50s', '4s', '6s',
      'First_Name', 'Last_Name', 'Cleaned_Highest_score',
      'Cleaned_Highest_Score', 'Boundary_Percentage'],
      dtype='object')
```

Q2. Top 2 players with maximum Average who have scored atleast 2 half centuries ?

```
In [148... qualified_players = df[df["50s"] >= 2]
top_avg_players = qualified_players.nlargest(2, "Avg")[["Player", "Avg"]]
print("Q2 Answer:")
print(top_avg_players)
```

Q2 Answer:

	Player	Avg
57	Deepak Hooda	101.0
60	Tom Curran	83.0

Q3 Create 2 new columns based on Player name. First column will have first name and second column will have last name. Eg: for the player Shikhar Dhawan, Shikhar will be the first name and Dhawan will be the last name

```
In [69]: # Ensure "Player" column is a string
df["Player"] = df["Player"].astype(str)

# Split player names into first and last names safely
df[["First_Name", "Last_Name"]] = df["Player"].str.split(n=1, expand=True)

# Check for missing values
print(df[["Player", "First_Name", "Last_Name"]].head())
```

	Player	First_Name	Last_Name
0	KL Rahul	KL	Rahul
1	Shikhar Dhawan	Shikhar	Dhawan
2	David Warner	David	Warner
3	Shreyas Iyer	Shreyas	Iyer
4	Ishan Kishan	Ishan	Kishan

Q4. Create a new column (Cleaned_Highest_score) based on Highest score variable.
Remove the Asterik(*) mark and convert the data type into INT.

```
In [67]: # Ensure "Highest Score" is a string and remove "*" properly
df["Cleaned_Highest_Score"] = df["Highest Score"].astype(str).str.replace(r"\*",
# Convert to numeric (handling NaN values)
df["Cleaned_Highest_Score"] = pd.to_numeric(df["Cleaned_Highest_Score"], errors=
```

```
In [54]: print(df[["Player", "Cleaned_Highest_score"]].head())
```

	Player	Cleaned_Highest_score
0	KL Rahul	132
1	Shikhar Dhawan	106
2	David Warner	85
3	Shreyas Iyer	88
4	Ishan Kishan	99

Q5. Print the total number of centuries scored in the entire season

```
In [57]: # Ensure "Runs" column is numeric
df["Runs"] = pd.to_numeric(df["Runs"], errors="coerce").fillna(0).astype(int)

# Count the number of innings where a player scored at least 100 runs
total_centuries = (df["Runs"] >= 100).sum()

print("Total number of centuries scored in the entire season:", total_centuries)
```

Total number of centuries scored in the entire season: 58

Q6. Print all the player names whose strike rate is less than the average strike rate of all players in entire season. Print the player name, his strike rate and average strike rate.

```
In [60]: average_strike_rate = df["Strike rate"].mean()
low_sr_players = df[df["Strike rate"] < average_strike_rate][["Player", "Strike
print("Q6 Answer:")
print(low_sr_players)
```

Q6 Answer:

	Player	Strike rate
51	Ajinkya Rahane	105.60
55	Glenn Maxwell	101.88
58	Vijay Shankar	101.04
61	Josh Philippe	101.29
62	Gurkeerat Singh	88.75
65	Kedar Jadhav	93.93
70	Yashasvi Jaiswal	90.90
71	Shreyas Gopal	94.87
77	Murali Vijay	74.41
79	Chris Jordan	93.54
80	Navdeep Saini	100.00
82	Kamlesh Nagarkoti	70.96
84	Harshal Patel	87.50
85	Jimmy Neesham	105.55
86	Tom Banton	90.00
89	Prabhsimran Singh	100.00
92	Kuldeep Yadav	61.90
94	Moeen Ali	75.00
95	Sandeep Sharma	80.00
96	Shardul Thakur	57.14
98	Rinku Singh	100.00
99	Shivam Mavi	71.42
100	Varun Chakaravorthy	66.66
101	Jaydev Unadkat	69.23
102	Ankit Rajpoot	90.00
104	Shahbaz Nadeem	87.50
105	Pravin Dubey	53.84
107	Deepak Chahar	58.33
108	Ravi Bishnoi	58.33
109	Andrew Tye	100.00
111	Kartik Tyagi	66.66
112	Murugan Ashwin	100.00
114	T Natarajan	60.00
115	Prasidh Krishna	50.00
116	Rahul Chahar	50.00
117	Mohammad Shami	66.66
118	Nikhil Naik	33.33
119	Mujeeb Ur Rahman	33.33
120	Dale Steyn	50.00
121	Varun Aaron	10.00
122	Shahbaz Ahmed	100.00
123	Yuzvendra Chahal	33.33
124	Mitchell Marsh	0.00
125	Umesh Yadav	0.00
126	Bhuvneshwar Kumar	0.00
127	Sheldon Cottrell	0.00
128	Khaleel Ahmed	0.00
129	Arshdeep Singh	0.00
130	Daniel Sams	0.00
131	Shreevats Goswami	0.00
132	Trent Boult	0.00

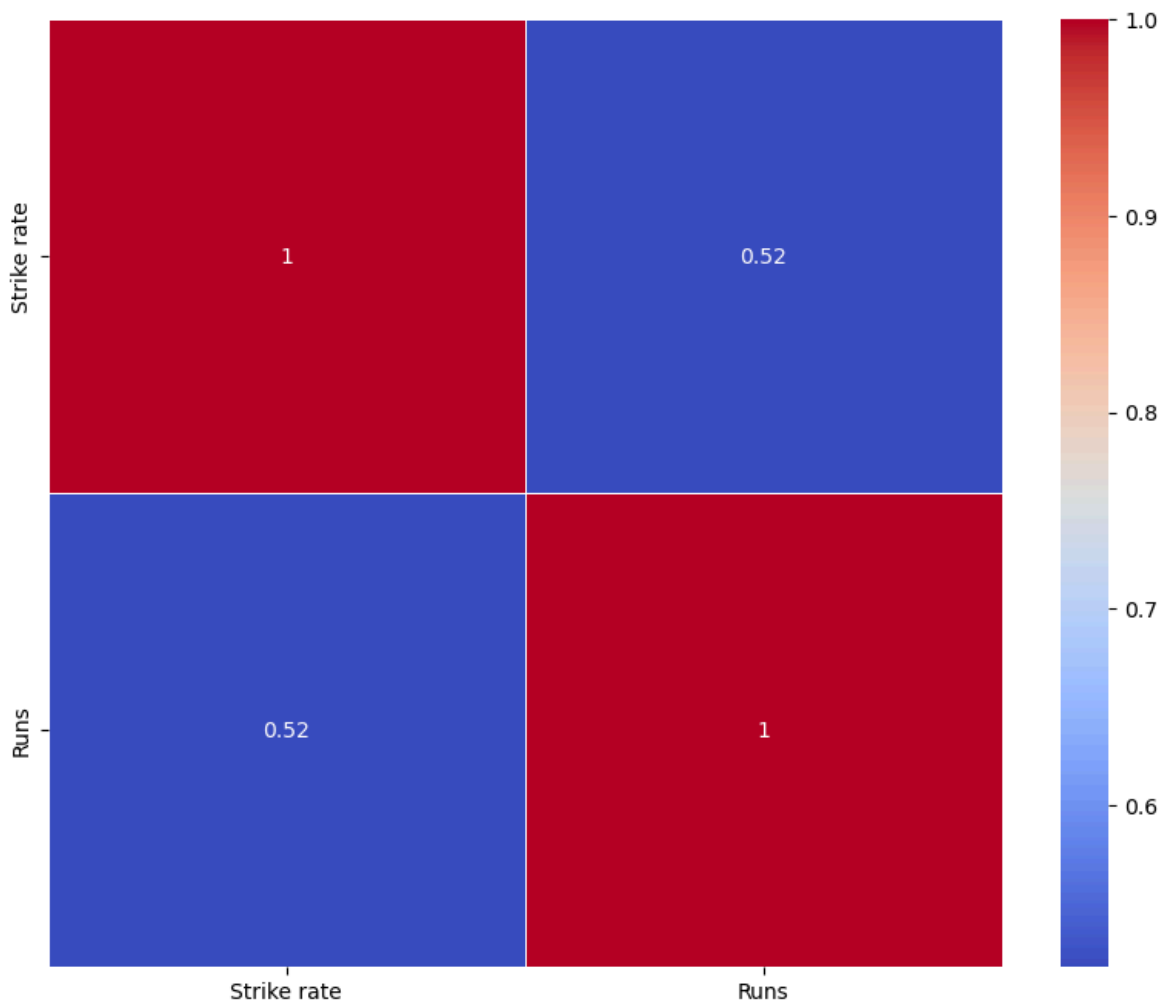
Q7. Please check the correlation between the features and create a heat map.

In [152...

```
correlation_matrix = df[['Strike rate', 'Runs']].corr()  
# Create a heatmap
```

```
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', linewidths=0.5)
```

Out[152... <Axes: >



Q8. Check the list of players who has an average greater than 50 as well strike rate above 120. Print player name, average and strike rate.

```
In [76]: high_perf_players = df[(df["Avg"] > 50) & (df["Strike rate"] > 120)][["Player",
print("Q8 Answer:")
print(high_perf_players)
```

Q8 Answer:

	Player	Avg	Strike rate
0	KL Rahul	55.83	129.34
4	Ishan Kishan	57.33	145.76
31	Kieron Pollard	53.60	191.42
36	Wriddhiman Saha	71.33	139.86
37	Ruturaj Gaikwad	51.00	120.71
57	Deepak Hooda	101.00	142.25
60	Tom Curran	83.00	133.87

Q9. Please check the list of players who has an average greater than 40 and balls faced above 100. Print player name, average and balls faced.

```
In [81]: experienced_players = df[(df["Avg"] > 40) & (df["Balls faced"] > 100)][["Player",
print("Q9 Answer:")
print(experienced_players)
```

Q9 Answer:

	Player	Avg	Balls faced
0	KL Rahul	55.83	518
1	Shikhar Dhawan	44.14	427
4	Ishan Kishan	57.33	354
8	Virat Kohli	42.36	384
9	ABD Villiers	45.40	286
10	Faf Duplessis	40.81	319
14	Eoin Morgan	41.80	302
24	Kane Williamson	45.28	237
27	Chris Gayle	41.14	210
28	Ben Stokes	40.71	200
31	Kieron Pollard	53.60	140
32	Rahul Tewatia	42.50	183
33	Ravindra Jadeja	46.40	135
36	Wriddhiman Saha	71.33	153
37	Ruturaj Gaikwad	51.00	169

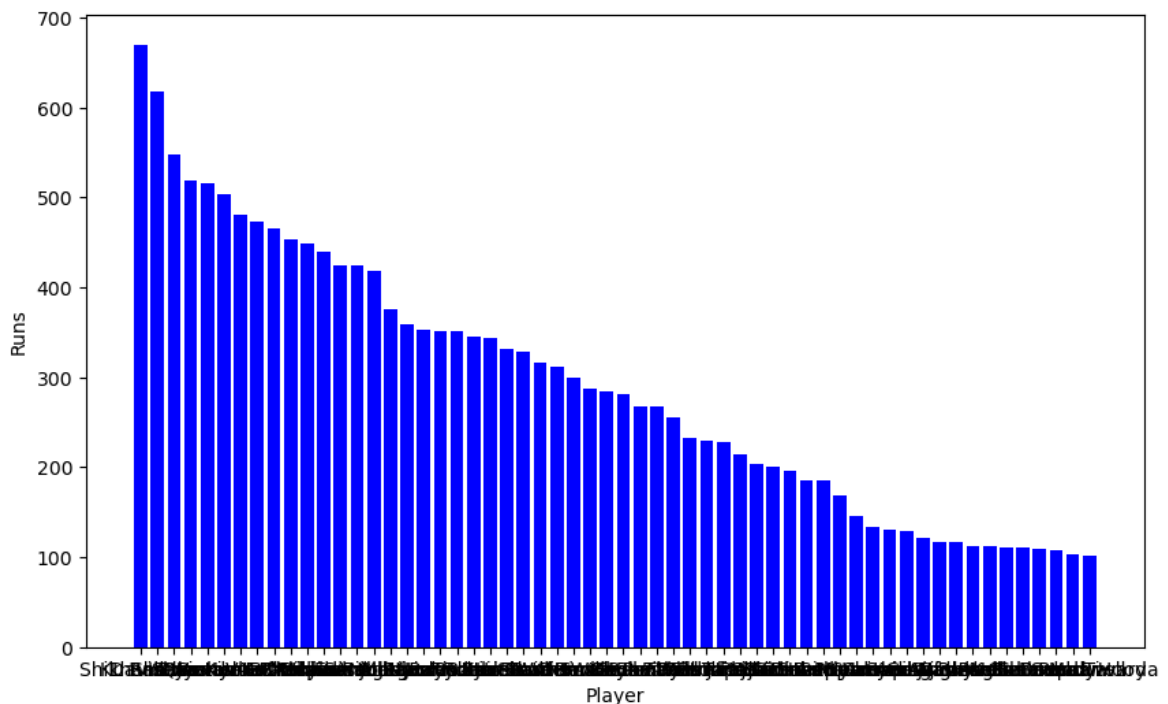
Q10. Players who scored atleast one century in this season. Create visualization.

```
In [86]: # Ensure "Runs" column is numeric
df["Runs"] = pd.to_numeric(df["Runs"], errors="coerce").fillna(0).astype(int)

# Filter players who have scored at Least one century
century_players = df[df["Runs"] >= 100]

# Visualization: Players who scored at Least one century
plt.figure(figsize=(10, 6))
plt.bar(century_players["Player"], century_players["Runs"], color="blue")
plt.xlabel("Player")
plt.ylabel("Runs")
```

Out[86]: Text(0, 0.5, 'Runs')



Q11. Players who scored atleast 4 half centuries in this season.

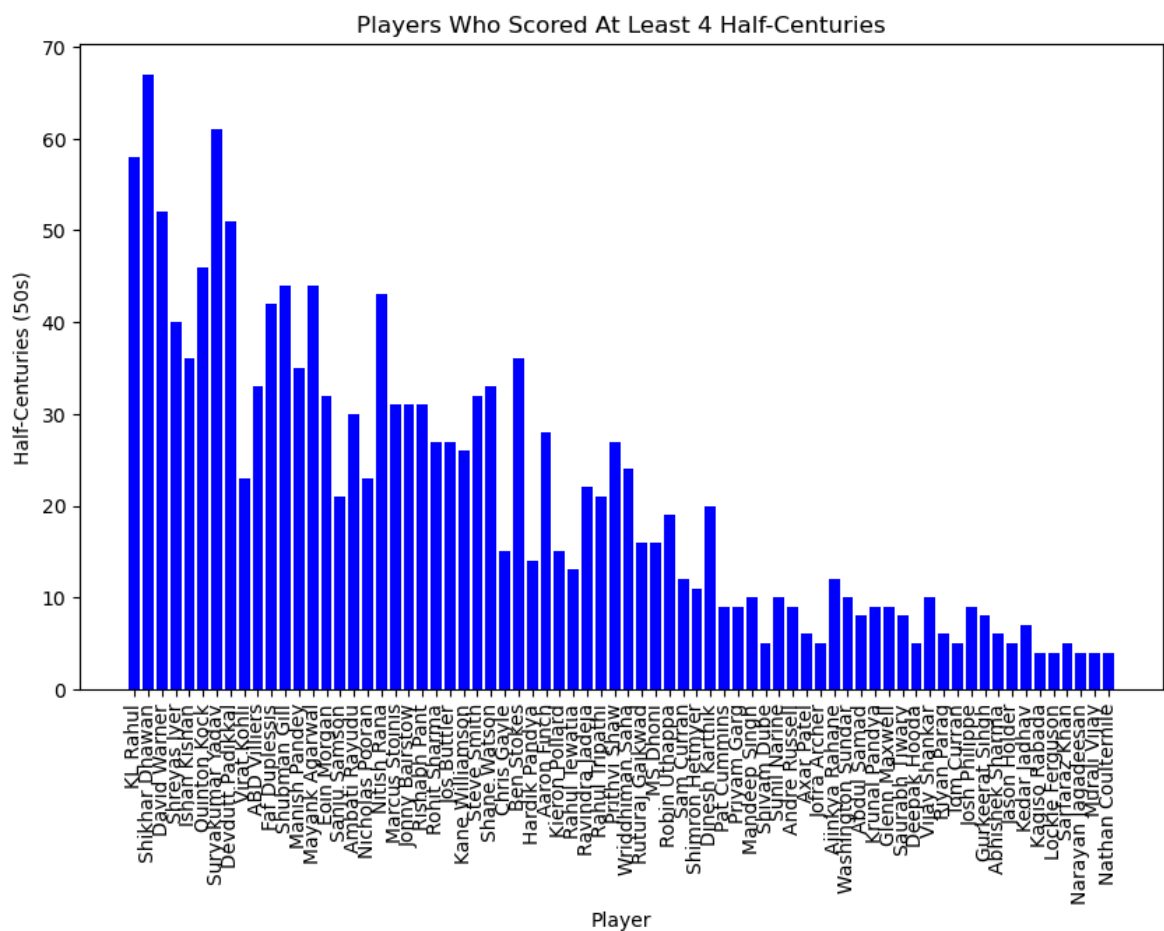
```
In [91]: print(df.columns[df.columns.duplicated()])
```

```
Index(['50s'], dtype='object')
```

```
In [93]: df = df.loc[:, ~df.columns.duplicated()]
```

```
In [162... # Ensure "50s" column is numeric
df["50s"] = pd.to_numeric(df["50s"], errors="coerce").fillna(0).astype(int)

half_century_players = df[df["50s"] >= 4]
plt.figure(figsize=(10, 6))
plt.bar(half_century_players["Player"], half_century_players["50s"], color="blue")
plt.xlabel("Player")
plt.ylabel("Half-Centuries (50s)")
plt.title("Players Who Scored At Least 4 Half-Centuries")
plt.xticks(rotation=90)
plt.show()
print("Players who scored at least 4 half-centuries in the season:")
print(half_century_players[["Player", "50s"]])
```



Players who scored at least 4 half-centuries in the season:

	Player	50s
0	KL Rahul	58
1	Shikhar Dhawan	67
2	David Warner	52
3	Shreyas Iyer	40
4	Ishan Kishan	36
..	...	...
68	Lockie Ferguson	4
75	Sarfaraz Khan	5
76	Narayan Jagadeesan	4
77	Murali Vijay	4
81	Nathan Coulterneile	4

[72 rows x 2 columns]

Q12. Check the list of players who hit more than 45 boundaries and more than 10 sixes in this season.

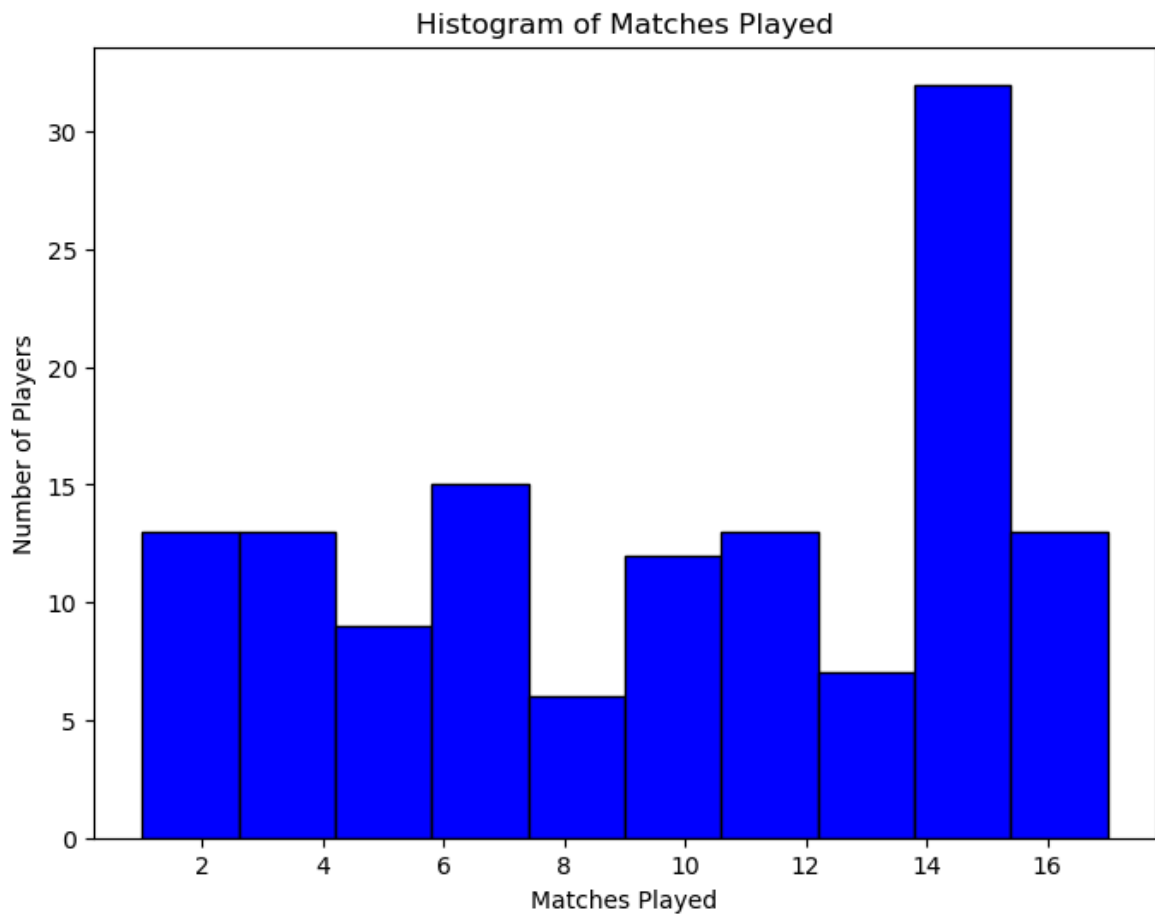
```
In [100... boundary_hitters = df[(df["4s"] > 45) & (df["6s"] > 10)][["Player", "4s", "6s"]]
print("Q12 Answer:")
print(boundary_hitters)
```

Q12 Answer:

	Player	4s	6s
0	KL Rahul	58	23
1	Shikhar Dhawan	67	12
2	David Warner	52	14
5	Quinton Kock	46	22
6	Suryakumar Yadav	61	11

Q13. Plot a histogram of number of matches played in a season by players.

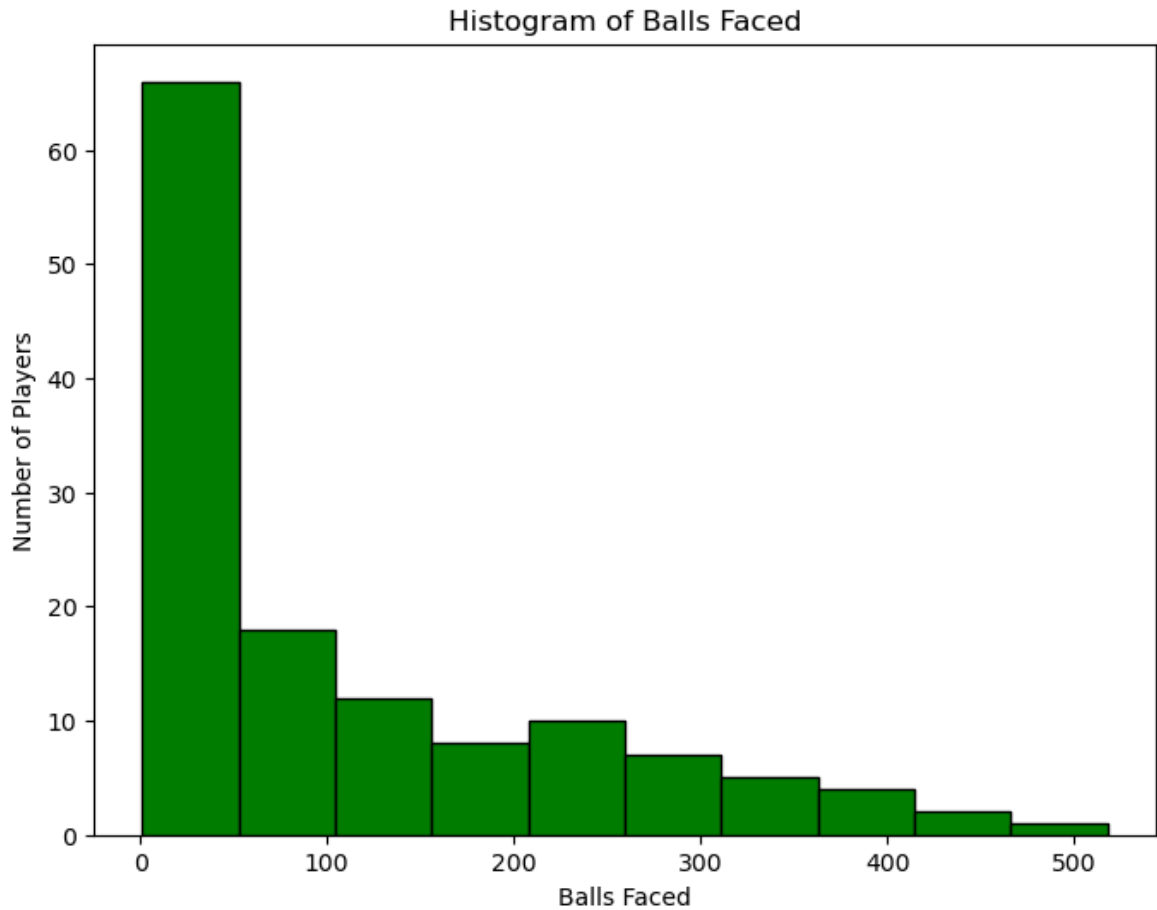
```
In [103... plt.figure(figsize=(8,6))
plt.hist(df["Matches"], bins=10, color='blue', edgecolor='black')
plt.xlabel("Matches Played")
plt.ylabel("Number of Players")
plt.title("Histogram of Matches Played")
plt.show()
```

Q14. Plot the histogram of balls faced by players

In [109...

```
plt.figure(figsize=(8,6))
plt.hist(df["Balls faced"], bins=10, color='green', edgecolor='black')
plt.xlabel("Balls Faced")
plt.ylabel("Number of Players")
plt.title("Histogram of Balls Faced")
plt.show()
```



Q15. Top 10 players with most runs in a season.

```
In [112... top_run_scorers = df.nlargest(10, "Runs")[['Player', 'Runs']]
print("Q15 Answer:")
print(top_run_scorers)
```

Q15 Answer:

	Player	Runs
0	KL Rahul	670
1	Shikhar Dhawan	618
2	David Warner	548
3	Shreyas Iyer	519
4	Ishan Kishan	516
5	Quinton Kock	503
6	Suryakumar Yadav	480
7	Devdutt Padikkal	473
8	Virat Kohli	466
9	ABD Villiers	454

Q16. Print the players who played the match but didn't get the batting.

```
In [164... players_no_batting = df[df["Inns"] == 0][["Player", "Matches"]]
print("Q16 Answer:")
print(players_no_batting)
```

Q16 Answer:

```
Empty DataFrame
Columns: [Player, Matches]
Index: []
```

Q17. Create a new column to show the percentage of total runs scored in 4s and 6s. Then print the top 5 players with maximum percentage

```
In [118... df["Boundary_Percentage"] = ((df["4s"] * 4 + df["6s"] * 6) / df["Runs"]) * 100
top_boundary_players = df.nlargest(5, "Boundary_Percentage")[['Player', 'Boundary_Percentage']]
print("Q17 Answer:")
print(top_boundary_players)
```

Q17 Answer:

	Player	Boundary_Percentage
109	Andrew Tye	100.000000
48	Andre Russell	76.923077
74	Chris Morris	76.470588
29	Hardik Pandya	73.309609
47	Sunil Narine	72.727273

Q18. Print the players with top 5 Not out percentages (Not Out percentage can be calculated as number of Not outs divided by Innings).

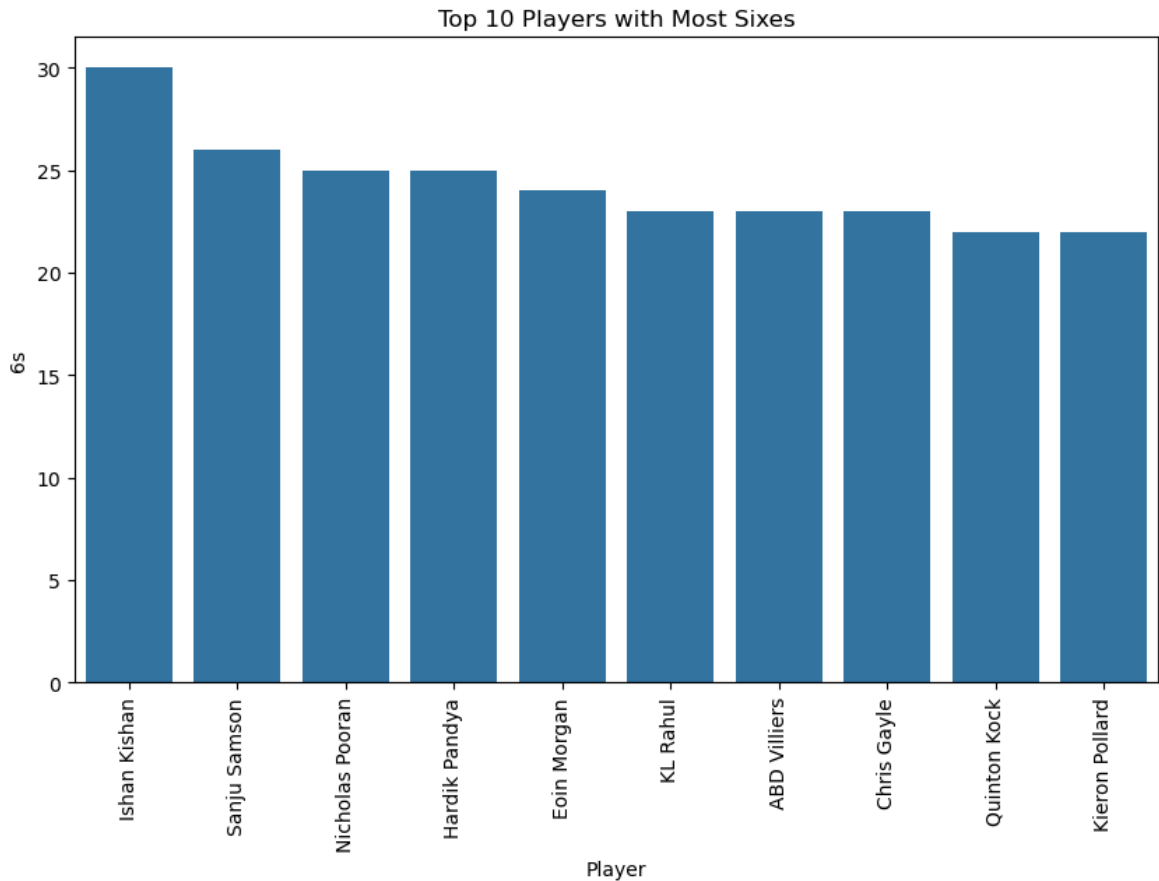
```
In [170... df["NotOut_Percentage"] = (df["Not Out"] / df["Inns"]) * 100
top_not_outs = df.nlargest(5, "NotOut_Percentage")[['Player', 'NotOut_Percentage']]
print("Q18 Answer:")
print(top_not_outs)
```

Q18 Answer:

	Player	NotOut_Percentage
62	Gurkeerat Singh	100.0
68	Lockie Ferguson	100.0
93	Imran Tahir	100.0
97	Mohammad Nabi	100.0
105	Pravin Dubey	100.0

Q19. Create visualization of top 10 players with highest number of sixes.

```
In [173... top_six_hitters = df.nlargest(10, "6s")
plt.figure(figsize=(10,6))
sns.barplot(x='Player', y='6s', data=top_six_hitters)
plt.xticks(rotation=90)
plt.title("Top 10 Players with Most Sixes")
plt.show()
```

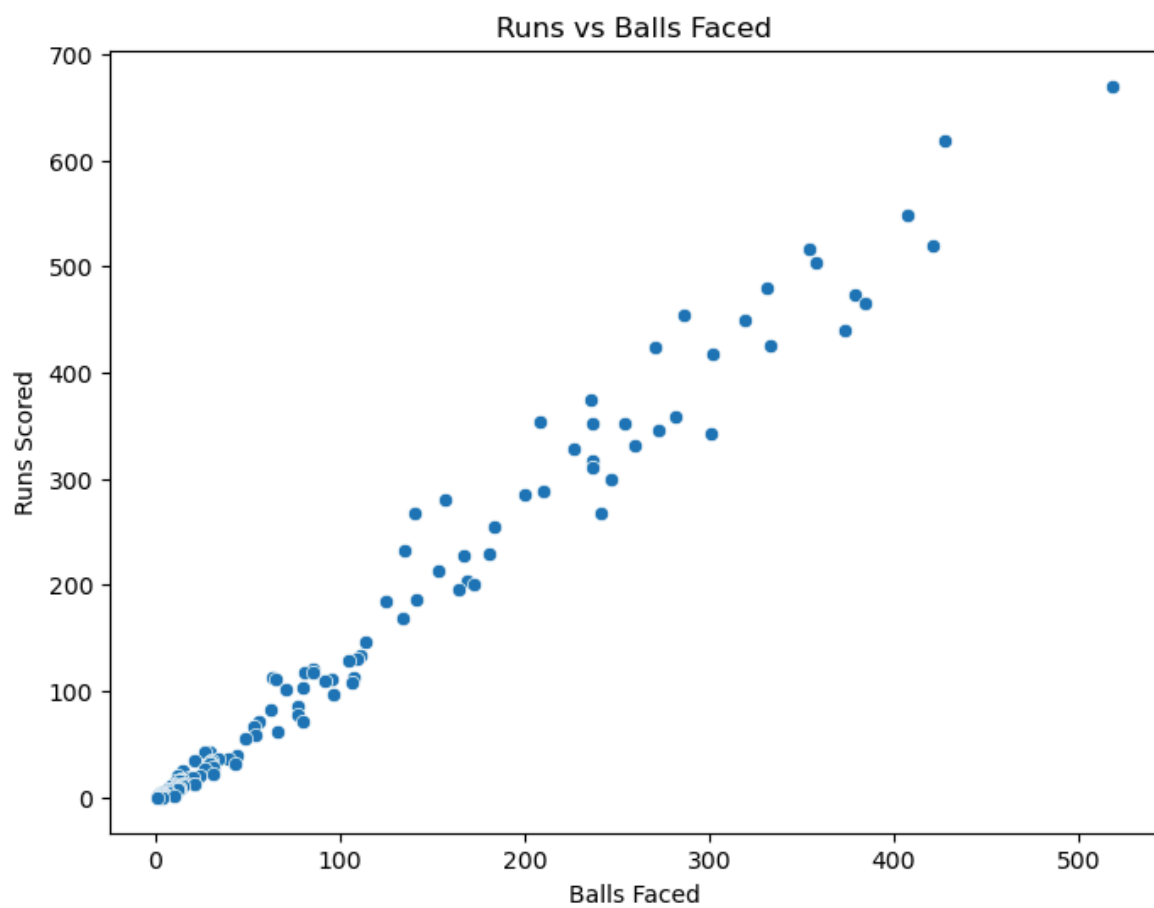


Q20. Scatter plot of runs scored by a player v/s balls faced in a season. Then find the relationship between these 2 variables.

In [178...

```
plt.figure(figsize=(8,6))
sns.scatterplot(x=df['Balls faced'], y=df['Runs'])
plt.xlabel("Balls Faced")
plt.ylabel("Runs Scored")
plt.title("Runs vs Balls Faced")
plt.show()

# Relationship between Runs and Balls Faced
correlation = df[['Balls faced', 'Runs']].corr().iloc[0,1]
print("Q20 Answer: Correlation between Runs and Balls Faced:", correlation)
```



Q20 Answer: Correlation between Runs and Balls Faced: 0.9899480233860832

In []: